

KAFKA HANDS-ON

Live demos inside a PDF file

A PDF has no HTML and no CSS — only form fields and its own JavaScript. The demos on the next pages are built out of them: log cells are buttons, motion is a timer, captions are text fields. Press the buttons, they are real.

append-only



VIEWER CHECK

Scripts are not running — this needs Chrome, Edge or Acrobat Reader

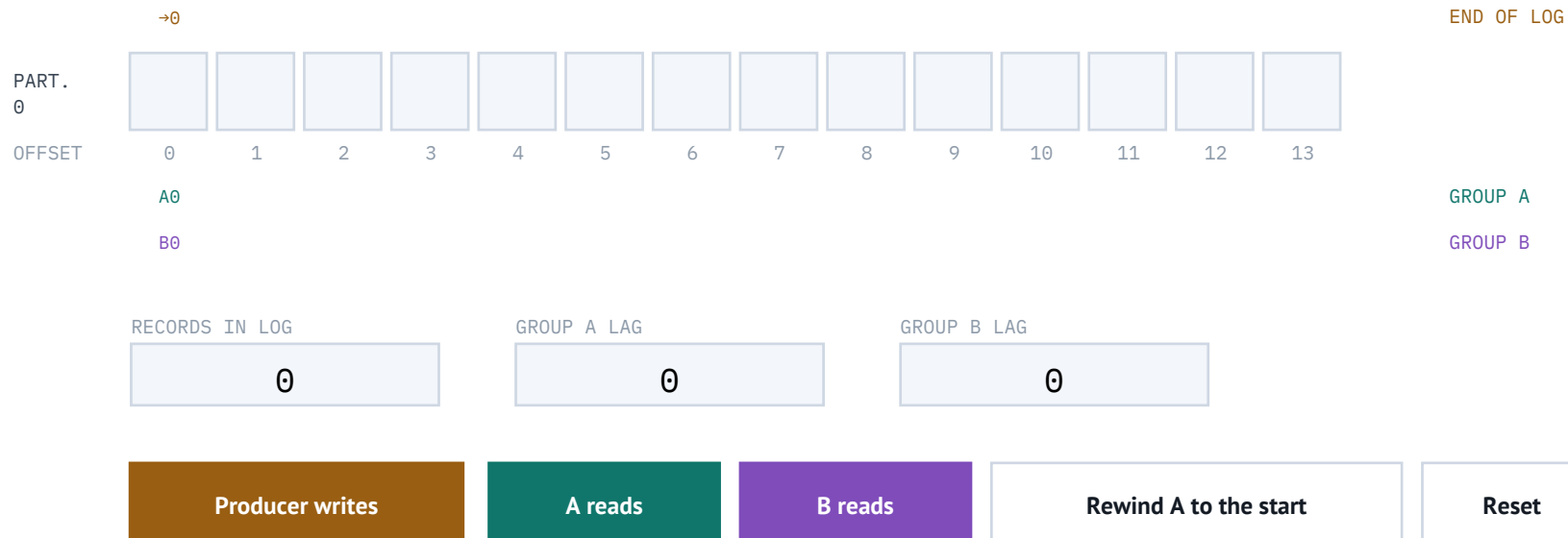
If the line above changed — everything works. If it stayed exactly as it is, the viewer cannot run PDF scripts: that is how Preview on macOS, the built-in readers on phones and the previews inside social apps behave. Download the file and open it in a browser on a computer.

Keep scrolling →

DEMO 01

A log, not a queue

Reading deletes nothing. The producer appends to the end, and every reader keeps a bookmark of its own and moves only that one. Hence replay, and hence independent consumers.



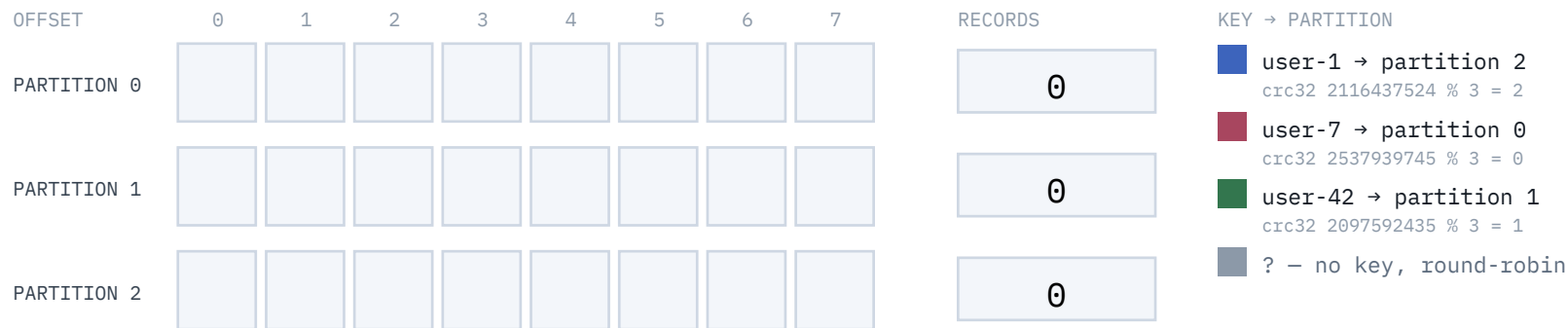
The demo runs by itself — press any button to take over

Notice: when A reads, the cell stays where it is — only the bookmark moves. The rewind button sends A back to the start, and the whole history is read again. A queue has no such button: there, whatever was read is gone.

DEMO 02

The key picks the partition

The producer doesn't choose a partition by hand — the key chooses instead: $\text{number} = \text{hash}(\text{key}) \% \text{partition count}$. What follows from that for record order, and what happens when one key drags everything onto itself?



HOW THE PARTITION WAS PICKED

$\text{hash}(\text{key}) \% 3 = \text{partition number}$

user-1

user-7

user-42

no key

Skew: user-42 six times

Reset

The demo runs by itself — press any button to take over

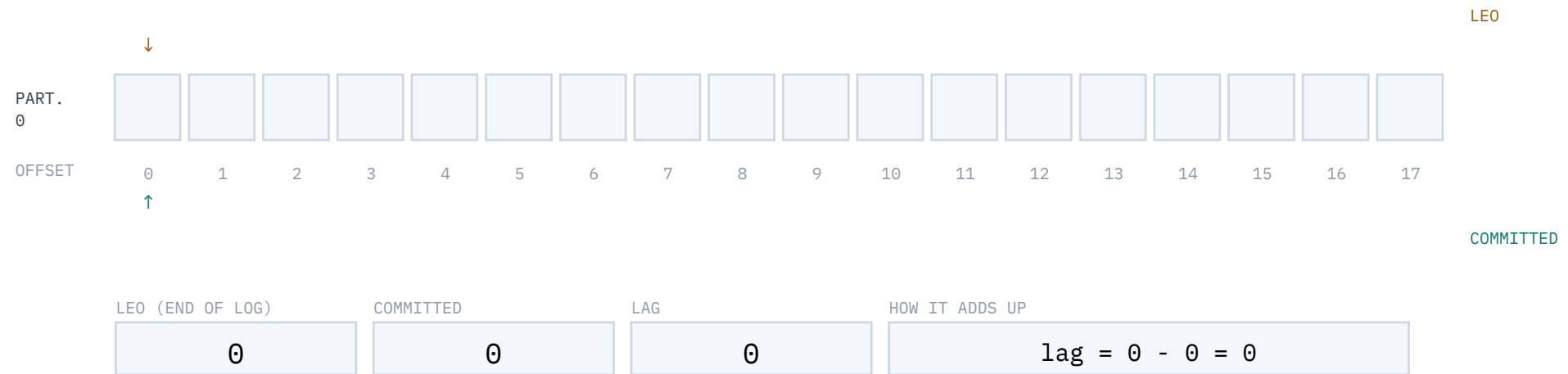
The hash is taken over the bytes of the key: as long as the partition count stays the same, a key always leads to the same partition. Add partitions and the divisor changes — the same key moves to another lane, while everything written earlier stays where it was. Different keys can land in one partition: the hash promises a stable address, not a distinct one. With no key the producer itself places the record — round-robin here, “sticky” batches in today's Kafka. Kafka's hash is murmur2, here it's crc32 for clarity: the arithmetic is the same.

Nothing moving? Open this file in Chrome, Edge or Acrobat Reader — those run the scripts a PDF carries.

DEMO 03

Lag: how stale your data is

Right now, how many messages has the consumer not read yet? Lag is the gap between the end of the log and the group's committed position, and it is what answers "how fresh is my data".



Producer faster

Consumer faster

Crash the consumer

Bring it back up

Reset

The simulation is already running: the producer writes one record per tick, the consumer reads one. Press any button to break the balance.

The committed offset is stored inside Kafka itself, in the internal topic `__consumer_offsets`, not in the consumer. That is why a crashed consumer comes back and carries on from the stored position instead of from zero. One more subtlety: committed is the number of the NEXT message to read, not of the last one processed. The strip here is a window onto 18 positions of an endless log: when the end of the log hits the right edge, the window slides right and the numbers under the cells slide with it – offset itself only grows, just as in Kafka. And once the lag outgrows the strip, the end of the log leaves the right edge: there is nowhere to draw it, but the numbers keep counting – lag has no ceiling until retention deletes the records.

Nothing moving? Open this file in Chrome, Edge or Acrobat Reader – those run the scripts a PDF carries.

DEMO 04

acks=all is deceptive without min.insync.replicas

The producer waits for “all replicas” to acknowledge — but only those in ISR right now, and that list can collapse down to the leader alone. How many copies does a record have once the producer has been told “ok”?

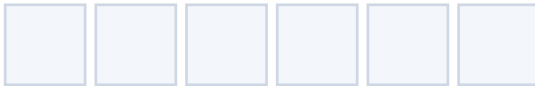
TOPIC ORDERS, PARTITION 0, REPLICATION.FACTOR = 3

BROKER 1
LEADER



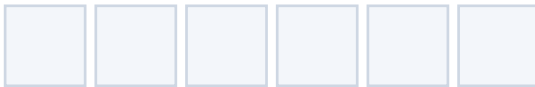
leader, alive

BROKER 2
REPLICA



in ISR

BROKER 3
REPLICA



in ISR

ACKS — WHAT THE PRODUCER WAITS FOR

0

1

all

all

MIN.INSYNC.REPLICAS — MINIMUM REPLICAS IN ISR

1

2

3

1

ISR — REPLICAS KEEPING UP WITH THE LEADER

ISR = [1, 2, 3]

the verdict shows up once you kill the leader

Write

Kill the leader

Replicas fall behind

Replicas catch up

Safe trio

Reset

The demo runs itself — press any button to take over

acks=all does not promise three copies — it promises to wait for whoever is in ISR right now. Let ISR collapse to the leader alone and “all” means one, yet the producer still gets its ok. An explicit refusal beats a silent loss: replication.factor = 3, acks = all, min.insync.replicas = 2.

Nothing moving? Open this file in Chrome, Edge or Acrobat Reader — those run the scripts a PDF carries.

THAT WAS THE SHORT VERSION

The full walkthrough – fifteen chapters in a browser

Partitions and keys, consumer groups and rebalances, retention and compaction, leader and ISR, acks and the silently lost write, lag diagnosis partition by partition, a sandbox and a cheat sheet for interviews. All of it is interactive, opens in any browser and works on a phone.

warhammer2000.github.io/kafka-na-palcah

And this PDF is a side experiment: the interactivity inside the document is made of form fields and document JavaScript, because a PDF has neither HTML nor CSS. The builder sources sit in the same repository.